SPRINT 02, QUALIDADE E CI

Qualidade de software e IA no processo

Hoje a aula é toda em torno de três exercícios em um repo real: abrir uma MR limpa, revisar uma MR com problemas plantados, triar apontamentos do Sonar. No fim, comparar com o que a IA vê no mesmo código. Pouco slide, muito teclado.

Aula	04 de 14
Data	07 de maio de 2026, quinta-feira, 10h00 às 12h30
Professor	José Romualdo da Costa Filho
Módulo	13, Engenharia de Software, Turma T13

Agenda da aula

inteli.

Sala invertida, 2h30, organizada em torno de um repo que vocês vão clonar e bater em cima. Quatro exercícios, três debriefs, um fechamento curto. Vocês escrevem mais MRs e mais comentários de revisão do que slides lidos.

Bloco	Atividade	Duração
10h00, Check-in	Recap dos auto-estudos, alinhar vocabulário do dia	15 min
10h15, Exercício A	Em dupla, abrir MR pequena e bem descrita em `feat/student-status`	35 min
10h50, Debrief A	Duas MRs no projetor, turma comenta em voz alta	10 min
11h00, Exercício B	Individual, revisar a MR suja em `feat/scholarship-check`	30 min
11h30, Debrief B	Agregar achados em um quadro comum	15 min
11h45, Exercício C	Em dupla, triar apontamentos do SonarQube	30 min
12h15, Exercício D	Mesma MR do B, mas pedindo à IA para revisar	10 min
12h25, Fechamento	Pontos de atenção e bridge para a Aula 05	5 min

CHECK-IN, 15 MIN

Check-in e recap dos auto-estudos

Três perguntas, mão levantada rápida. Qual característica da ISO 25010 mais apareceu no seu projeto? O que te incomodou no Google Engineering Practices? A IA te ajudou ou atrapalhou na última MR que você abriu?

15:00

INICIAR PAUSAR ZERAR

Objetivos de aprendizagem

inteli.

Quatro verbos. A ideia não é sair sabendo o que é ISO 25010 de cor. É sair com o músculo de escrever uma MR que o revisor agradece, de ler código dos outros sem virar carimbeiro, de olhar o painel do Sonar sem pirar, e de saber onde a IA cobre bem e onde ainda engana.

Abrir

Montar uma MR pequena, com descrição que explica o porquê, teste que prova o caso e commit claro.

Revisar

Ler um diff dos outros aplicando a rubrica blocker, question, nit. Sem ir para o pessoal.

Triar

Olhar dez apontamentos do Sonar e decidir quais consertar agora, virar issue, ou justificar o descarte.

Comparar

Colocar lado a lado o que o revisor humano pegou e o que a IA pegou, ver onde um cobre o outro.

A aula é invertida, e ela cobra isso

Quem pulou os artigos do Google e do GitHub vai sentir no exercício B. Não dá para improvisar rubrica de revisão no meio do exercício. Se perdeu o auto-estudo, pega a pessoa ao lado antes do bloco 2 começar.

Qualidade é invisível, até ela quebrar

inteli.

Ninguém abre o console e pensa "hoje quero um código bonito". As decisões de qualidade entram escondidas: no momento em que você escreve o MR, no momento em que você comenta o MR do colega, no momento em que o pipeline roda Sonar e você escolhe se olha o resultado ou fecha a aba. A aula de hoje torna esses três momentos visíveis, porque enquanto invisíveis eles viram dívida técnica silenciosa.

Qualidade que aparece no painel

- Cobertura de testes.
- Bugs fechados por sprint.
- Tempo médio de build.
- Fácil de medir, fácil de gamear.

Qualidade que só aparece na rotina

- Descrição de MR que poupa uma pergunta.
- Comentário que ensina em vez de humilhar.
- Refatoração que não virou MR gigante.
- Difícil de medir, difícil de fingir.

> *"Code review is not about catching bugs. It is about everyone on the team getting to see how the code evolves."*
>
> Google Engineering Practices, adaptado

ISO 25010 em uma tela, só para alinhar

inteli.

Vocês leram a norma inteira no auto-estudo. O lembrete aqui é para deixar as oito famílias à vista. Hoje a gente vive em duas delas, manutenibilidade e segurança. As outras seis não somem, só não são o foco desta aula.

Functional suitability
Faz o que promete, com exatidão.

Performance efficiency
Tempo de resposta aceitável.

Compatibility
Convive com outros sistemas.

Usability
Aprendível, difícil de usar errado.

Reliability
Funciona, se recupera de falha.

Security
Foco da aula. Protege dado e acesso.

Maintainability
Foco da aula. Lê-se e altera-se sem medo.

Portability
Roda em outros ambientes.

Por que o foco em manutenibilidade

A norma quebra manutenibilidade em cinco subatributos: modularidade, reusabilidade, analisabilidade, modificabilidade, testabilidade. Revisão de código, análise estática e IA na revisão mexem direto nesses cinco. Por isso são o coração da aula.

Três atores olham seu código

inteli.

A aula inteira gira em torno destes três. Cada um tem um ponto cego diferente. A qualidade boa do time aparece quando os três conversam, não quando um tenta fazer o papel dos outros.

ATOR 01

Revisor humano

Pega contexto de domínio, histórico do time, decisão de produto. Erra por cansaço e por viés. Cobre o que ferramenta não vê.

ATOR 02

Análise estática

Pega padrão sintático, métrica objetiva, regra conhecida. Nunca cansa, nunca vê ironia. Cobre o que o humano esquece no cansaço.

ATOR 03

IA generativa

Pega padrão de código visto em milhões de repos, explica em linguagem natural. Inventa referência, alucina fix. Útil como segunda opinião, não como árbitro.

O perigo dos três

Se o time delega revisão só para a IA, a primeira MR com um bug de domínio passa batido. Se delega só para o Sonar, a primeira MR com decisão de arquitetura ruim passa batido. Se delega só para o humano, a fadiga silencia o que ele mesmo sabe que deveria cobrar. Três atores, três papéis, não substituíveis.

BLOCO 1, 90 MIN

Revisão de código, dois papéis

Primeiro vocês são autores de uma MR. Depois, revisores de uma MR suja. Os dois lados da mesma mesa.

Por que a gente revisa, de verdade

inteli.

A resposta clichê é "para pegar bugs". Se fosse só isso, análise estática já bastaria. Revisão serve para três coisas que ferramenta nenhuma entrega: transferir conhecimento, pressionar a qualidade do código para cima, e criar um registro do porquê das decisões. O bug pego é bônus.

Transferir conhecimento

Todo MR aberto é uma aula curta para quem não mexeu naquela parte. O revisor aprende o contexto, o autor ensina a escolha.

Pressionar qualidade

Saber que alguém vai ler te faz escrever melhor. É o efeito platéia do próprio time, sem precisar de meta.

Registrar decisões

Comentário no MR fica gravado. Seis meses depois, ninguém lembra por que o método ficou complexo. O thread do MR lembra.

O mito do carimbador

Time que aprova tudo em três minutos não faz revisão, faz certificação simbólica. Time que trava tudo por detalhe tampouco. Revisão boa tem atrito proporcional ao risco da mudança. Hotfix minúsculo passa em cinco minutos. Refatoração que toca dez arquivos merece trinta.

Como autor, quatro hábitos caros

inteli.

Resumo do que o Gergely Orosz escreve na Pragmatic Engineer e do que a própria documentação interna de grandes engenharias repete: a maioria das MRs ruins não é por código ruim, é por empacotamento ruim. Quatro hábitos que custam pouco para o autor e poupam muito do revisor.

1, Escopo pequeno

Uma MR, uma intenção. Separar refactor de feature. Se a descrição precisa de mais de dois parágrafos para explicar, quase sempre são dois MRs.

2, Descrição que responde o porquê

O diff mostra o que mudou. O MR precisa dizer por que mudou, o que NÃO mudou por decisão, e como testar em 30 segundos.

3, Teste que prova o caso

Um teste por caminho novo. Um teste negativo por erro esperado. Se o teste parece trivial, ótimo, ele vira documentação executável.

4, Revisar o próprio diff antes de pedir revisão

Abra o MR no navegador e leia linha por linha. Metade dos comentários dos outros some aí, porque você mesmo pega.

A heurística dos 400 linhas

Estudo clássico da SmartBear mostra que a qualidade da revisão cai depois de 400 linhas de diff. Acima de 800, o revisor faz leitura dinâmica sem querer. Se seu MR passou de 400, considere dividir. Revisor cansado é pior que revisor ausente, porque ele aprova achando que leu.

Como revisor, a linha do Google

inteli.

O Google Engineering Practices é o documento mais citado quando se fala em revisão de código. A versão curta: a meta não é chegar no código perfeito. É chegar no código **melhor do que estava** sem travar o time. Três regras que valem mais que vinte.

Prefira progresso

Se o MR deixa o código melhor, aprove. Perfeição paralisa. "Not worse" é critério real.

Princípios, não gosto pessoal

Cada comentário deve apontar um princípio (legibilidade, consistência, performance). Gosto próprio vira "eu preferiria", e isso é nit, não blocker.

Responda em até um dia útil

Revisão parada é fricção amplificada. O autor perde contexto, a MR vira patchwork. Atrasar 3 dias custa mais ao time que aprovar com ressalva.

Tom profissional, sempre

Comentário de revisão é registro permanente. Evite "isso está errado", prefira "isso pode gerar X quando Y". Critique o código, não a pessoa. Em caso de discordância insolúvel, escale para plenária ou PR, nunca resolva no thread em tom de duelo.

A rubrica que vocês vão usar hoje

inteli.

Três rótulos, nada mais. Todo comentário que você fizer em uma MR hoje precisa entrar em uma dessas três caixas. A escolha do rótulo força o autocontrole: é mesmo blocker, ou eu só queria outra coisa?

BLOCKER

Não pode fundir

Bug claro, falha de segurança, quebra de contrato, teste ausente em código crítico. Se passar, alguém paga. Exige correção antes de aprovar.

QUESTION

Explica melhor

Não bloqueia, mas precisa de resposta. Decisão de design, trade-off, escolha que não está óbvia no diff. Autor responde, todos aprendem.

NIT

Opcional

Nome de variável, quebra de linha, preferência estilística sem regra formal. Autor pode aceitar ou ignorar sem justificativa longa.

Tip de revisor sênior

Se a MR só tem nits, aprove. Se tem 20 nits e zero blocker, pergunte a si mesmo se você está lendo o código ou fazendo linting manual. Lint automático resolve nit em escala. Revisão humana rende quando foca em question e blocker, onde a máquina não chega.

MR pequena, o superpoder

inteli.

Duas MRs, mesma feature, uma bem empacotada, outra não. A diferença de tempo de revisão e taxa de defeito pós merge é absurda. O que parece "mais esforço para dividir" no momento de empacotar vira menos esforço para todo mundo no resto do ciclo.

MR EMPACOTADA	
Linhas de diff	120
Commits	3, tipados
Descrição	Porquê, Como testar
Tempo médio de review	20 min
Rodadas de feedback	1 a 2
Defeito pós merge	raro

MR GIGANTE	
Linhas de diff	1200
Commits	1, "wip"
Descrição	vazia ou "ajustes"
Tempo médio de review	3 a 4 dias
Rodadas de feedback	5+
Defeito pós merge	esperado

Em duplas, resposta de cabeça

Pensem em um MR que vocês abriram na Sprint 01. Qual dos dois painéis ele mais se pareceu? Compartilhem em dupla, 30 segundos, antes do próximo slide. Honestidade conta mais que pose.

EXERCÍCIO A, EM DUPLA, 35 MIN

Abra sua primeira MR bem empacotada

Cada dupla clona o `academic-tracker` em github.com/josercf/inteli-2026-1B-T13-ES10-S3, cria a branch `feat/student-status`, implementa `GET /students/:id/status` e abre a MR. Regras:

1, Branch e commits

Nome `feat/student-status`. Pelo menos dois commits em Conventional Commits, um `feat` e um `test`.

`commit de "wip" não passa`

2, Teste

Um teste do caminho feliz (aluno existe), um do 404 (aluno não existe). Ambos verdes no CI.

`supertest já está no repo`

3, Descrição do MR

Use o template. Preencha "o que muda", "motivação", "como testar". Descrição vazia é motivo de não review.

`1 parágrafo por seção basta`

4, Auto-revisão

Antes de pedir review do par, leia seu próprio diff no navegador. Comente dúvidas suas na MR como se fosse outra pessoa.

`revisar o próprio é 80% do ganho`

DEBRIEF EM 35 Escolho duas MRs no projetor, lemos em voz alta, a turma comenta. Critério: uma boa e uma com algo a melhorar. Sem

Debrief A, o que olhar nas duas MRs

inteli.

Vou abrir duas MRs no projetor. A turma fala primeiro, em ordem: impressão de quem nunca viu, pergunta que o título deixou na mesa, o que faltou na descrição, se o teste prova o caso. Eu só sintetizo no fim.

Sinal de MR boa

Você lê a descrição e já tem hipótese do que vai ver no diff. Ao abrir o diff, confirma em vez de descobrir.

Sinal de MR a melhorar

A descrição repete o título. O diff traz três mudanças que não estavam prometidas. O teste existe mas cobre o caminho feliz e nada mais.

A pergunta de fechamento

Se você fosse revisar isso às 17h de uma quinta-feira, aprovaria, comentaria ou largaria para amanhã? Essa resposta conta mais que a rubrica.

O que vocês levam daqui

O debrief não é avaliação. É espelho. Mesmo quem teve MR escolhida vai reconhecer nela algo que o par escreveu antes. O ganho é sair do "tá bom assim?" para "eu sei o que falta, e sei por quê".

EXERCÍCIO B, INDIVIDUAL, 30 MIN

Revisar `feat/scholarship-check`

A MR já está aberta no repo, com o título "Ajuste bolsa" e descrição vazia. Eu plantei doze problemas de propósito. Sua missão: pegar pelo menos oito, classificados em blocker, question, nit.

1, Leitura

Leia a descrição, depois o diff. Anote toda vez que você precisou parar para entender o que está sendo feito. Essa fricção é sinal.

`descrição vazia já é achado`

2, Comentários no MR

Abra um thread por problema. Comece o comentário com [blocker], [question] ou [nit] para forçar o rótulo.

`um thread por problema, não combinar`

3, Rubrica mínima

Meta: pelo menos 2 blockers, 2 questions, 2 nits. Se tudo te parece blocker, releia, quase nunca é.

`perfeccionismo vira ruído`

4, Veredito

No fim, escreva "aprova", "aprova com nits", ou "pede mudanças". A decisão resume a leitura.

`sem veredito, não é revisão`

Debrief B, os doze que eu plantei

inteli.

A gente agrega o que a turma achou em um quadro comum. A lista abaixo é a que um revisor sênior pega sem suar. Se o seu número ficou perto dela, a rubrica entrou. Se ficou longe, não é drama: é o ganho da próxima semana.

BLOCKER, 5

Não pode fundir

Credencial default `'admin123'`. Rota sem teste. Magic number 8. Sem validação de entrada. Sem 404 para aluno inexistente.

QUESTION, 2

Explica melhor

Por que duplicar a lógica de média ponderada em vez de importar? Por que `==` se a base toda usa `===`?

NIT, 5

Opcional, mas pega

`console.log` sobrando. Commit "ajuste". Descrição vazia. Import morto (`buildError`). Função chamada `calc`.

Bônus, se alguém foi longe

Algum de vocês pegou o `// eslint-disable-next-line no-unused-vars` mascarando a variável `_token`? Esse é o sinal de revisor que lê pragma, não só código. Lint silenciado é lint escondido. Costuma ser o lugar onde o autor já sabia que tinha algo errado.

BLOCO 2, 30 MIN

Análise estática, o painel

O Sonar ativa, o painel enche. Seu trabalho não é consertar tudo, é decidir o que entra na fila.

O que análise estática vê, e o que não vê

inteli.

A ferramenta lê o código sem rodar. Olha sintaxe, padrão, relação entre trechos. Não sabe seu domínio, não sabe o porquê de cada escolha. O que ela pega e o que ela ignora desenha o papel que ela tem no time.

Pega bem

Credencial em claro, complexidade cognitiva alta, comparação frouxa (`==`), duplicação, dead code, código inalcançável.

Pega mais ou menos

Nomeação, bug lógico sutil, concorrência. A ferramenta dá sinal, mas sem contexto o sinal confunde mais do que ajuda.

Não pega

Decisão de arquitetura errada, regra de negócio invertida, trade-off de experiência do usuário. Só quem conhece o produto pega.

A pergunta que decide se o alerta vale

"Se isso fica assim por seis meses, o que quebra?" Se a resposta é "nada", é candidato a fila ou descarte. Se a resposta é "alguém paga", blocker. Análise estática vira útil quando o time aprende a responder essa pergunta sem romantizar nem descartar no automático.

SonarQube Community, SonarLint, e como se conectam

inteli.

O repo já vem com Sonar pronto para rodar localmente via Docker Compose. Sobe em três comandos. No pipeline, a gente só liga se `SONAR_HOST_URL` e `SONAR_TOKEN` estiverem definidos, senão o job fica dormindo. É de propósito, para não travar quem quer só rodar o repo sem subir o servidor.

SonarQube

Servidor que acumula análises no tempo. Painel único por projeto, histórico, QG (quality gate), comparação de branch.

SonarLint

Plugin da IDE. Mesmas regras do SonarQube, mas na hora de digitar. Pega o alerta antes de virar commit.

Sonar na pipeline

O CI roda scanner em cada push, publica no servidor. Se o QG falha, a pipeline falha. Sinal visível para o time inteiro, não só para quem abriu o MR.

```
npm run sonar:up        # sobe Sonar + Postgres em http://localhost:9000
npm run sonar:scan      # roda o scanner com coverage
npm run sonar:down      # derruba os containers quando terminar
```

Smells plantados no `academic-tracker`

inteli.

Eu plantei cerca de dez apontamentos de propósito no `main`, entre segurança, code smell, dead code e complexidade. Esta tela é preview, o Sonar devolve mais contexto quando vocês rodarem.

Arquivo	Severidade	Apontamento esperado
`src/config/index.js`	Blocker	credencial hardcoded, `jwtSecret` e `adminPassword`
`src/utils/validators.js`	Major	`==` em vez de `===`
`src/utils/validators.js`	Major	cognitive complexity alta em `classifyScore`
`src/utils/validators.js`	Minor	reatribuição de `label` após retorno (dead code)
`src/grades/grades.service.js`	Major	duplicação entre `computeSimpleAverage` e `computeWeightedAverage`
`src/grades/grades.service.js`	Minor	condição `totalCredits == 0` inalcançável
`src/app.js`	Minor	`console.error` e `console.log` em rota de produção

O que fazer com isso

A tabela é o mapa. No exercício C a missão é **triagem**, não consertar tudo. Três decisões possíveis por item: consertar agora, abrir issue, ignorar justificando.

EXERCÍCIO C, EM DUPLA, 30 MIN

Triagem de SonarQube

Com o Sonar rodando, vocês têm 10 apontamentos a triar. Em dupla, preencham a tabela e abram UMA MR consertando o item que vocês escolheram como "consertar agora".

1, Tabela

Colunas: severidade, arquivo, regra, o que diz, sua decisão, motivo. Decisão em três: consertar agora, abrir issue, ignorar justificando.

`sem motivo escrito, não vale`

2, Critério de triagem

Quatro filtros: risco real, custo de corrigir, dívida técnica, contexto. Nenhum alerta é verdade absoluta.

`alerta sem contexto vira ruído`

3, MR de exemplo

Escolham um apontamento da coluna "consertar agora" e abram MR corrigindo. Aplica tudo do Exercício A: escopo, descrição, teste.

`uma MR, uma intenção`

4, Entrega

Tabela preenchida, link do MR, três linhas sobre o que o painel mostrou que você não teria percebido só lendo código.

`o insight conta mais que o fix`

DEBATE CURTO AO Pergunta aberta para plenária: teve algum alerta que vocês descartaram com convicção? Qual e por quê? Justificar descarte é

Três decisões, quatro filtros

inteli.

Triagem não é lista de desejos. É método. Cada alerta passa pelos quatro filtros, depois entra em uma das três gavetas. Quando o time combina isso, o painel do Sonar deixa de assustar e vira ferramenta de fila.

DECISÃO 1

Consertar agora

Risco alto ou custo baixo de correção. Credencial em claro. Condição inalcançável. Typo em nome público. Fica barato e evita vergonha.

DECISÃO 2

Abrir issue

Vale corrigir, mas não agora. Requer escopo próprio, contexto, possivelmente MR só para isso. Ex.: refatorar `classifyScore`, remover duplicação de médias.

DECISÃO 3

Ignorar, justificando

Alerta não se aplica ao contexto. Ex.: `console.log` em script de CLI de dev, não em rota de produção. Ignorar exige comentário, não silêncio.

A armadilha da fila infinita

Time que abre issue para tudo sem fechar nenhuma está empurrando dívida disfarçada. Existe meta: do total triado, pelo menos 20% vira "consertar agora", pelo menos 30% vai para issue com dono e prazo, o resto é ignorado com justificativa escrita. Sem meta, o painel cresce e ninguém olha mais.

BLOCO 3, 15 MIN

IA na revisão

Mesma MR do exercício B, mas olhada por um modelo. Compara, discorda, cruza.

Três momentos da IA no ciclo

inteli.

A IA entra no fluxo de engenharia em três lugares diferentes. Cada um tem um ponto cego próprio, e cada um exige um hábito de uso diferente. Quem joga a IA em todos os três sem perceber a diferença fica refém.

1, Sugestão in-editor

Copilot, Cursor, Codeium. Completa enquanto você digita. Ganho de velocidade em boilerplate. Risco: aceitar sem ler.

2, Revisão automática de MR

GitLab Duo, Copilot Code Review, Graphite AI. Comenta no diff assim que a MR abre. Ganho de cobertura. Risco: o time parar de revisar achando que ela revisou.

3, Conversa com o código

Claude, ChatGPT, Gemini. Você cola o diff e pergunta. Ganho de segunda opinião. Risco: alucinação de referência e de arquivos que não existem.

A regra que eu uso

Os três são úteis, nenhum é árbitro. Sugestão in-editor, aceito só o que eu consigo explicar em voz alta. Revisão automática de MR, trato como mais um revisor, não como aprovador final. Conversa com o código, uso para abrir perspectiva, nunca para produzir fact sem checar.

IA na revisão, onde cobre e onde falha

inteli.

É o que eu vejo em revisões reais e o que aparece em estudos do último ano sobre uso de IA no fluxo de engenharia. Os pontos fortes e fracos não são fixos, mudam de modelo para modelo. Mas o padrão se repete o suficiente para virar heurística prática.

Onde a IA cobre bem

- Padrão sintático, convenção de estilo.
- Sugestão de nome alternativo.
- Detecção de duplicação próxima.
- Explicação de função antiga para quem acabou de chegar.
- Escrita de teste para caminho feliz.

Onde a IA falha

- Contexto de domínio fora do texto do prompt.
- Decisão de arquitetura baseada em histórico do time.
- Trade-off de experiência de usuário.
- Detectar lint silenciado disfarçado de intenção.
- Inventar arquivo, função ou referência que não existe.

A parábola da MR do B

Quase sempre a IA pega `console.log`, `==`, import morto, magic number. Quase nunca pega que o `eslint-disable` mascara a variável `_token`, nem questiona por que o `ADMIN_TOKEN` virou default `'admin123'`. O sinal de alerta em um lint silenciado exige conhecer a dinâmica de uma equipe. A IA não estava na sua daily.

EXERCÍCIO D, INDIVIDUAL, 10 MIN

IA revisa o mesmo MR do B

Pegue a MR `feat/scholarship-check` e peça à IA de sua escolha para revisar usando a mesma rubrica (blocker, question, nit). Compare com o que você pegou sozinho.

1, Escolha a ferramenta

GitLab Duo na pipeline, Copilot Code Review no PR, Claude/ChatGPT/Gemini no chat colando o diff. Qualquer das três serve.

`mesmo prompt em todas, se der`

2, Tabela de comparação

Colunas: ponto apontado, origem, humano pegou, IA pegou, quem acertou mais.

`mesmo ponto em linhas separadas`

3, Classifique cada ponto

Convergente, complementar, ruído da IA, ângulo morto humano. Quatro etiquetas, nada mais.

`sem meio-termo, decide`

4, Retorno para a plenária

Em uma ou duas linhas: onde a IA foi mais útil? Onde você ainda confia mais no revisor humano? Qual comentário te surpreendeu?

`honestidade, não torcida`

Três atores, três papéis, nenhum substituto

inteli.

A aula inteira cabe nesta tela. Humano, ferramenta estática, IA. Cada um tem um dia, cada um tem um cego. O trabalho de engenharia de qualidade é orquestrar os três, não escolher um campeão.

HUMANO

Contexto

Olha domínio, história do time, decisão de produto. Prioriza. Cansa. Precisa de MR pequena para render bem. Insubstituível em question e blocker de arquitetura.

SONAR

Consistência

Varre o repo inteiro sempre, sem cansar. Pega padrão sintático, segurança, duplicação, complexidade. Exige triagem humana para virar fila.

IA

Segunda opinião

Explica em linguagem natural, sugere alternativa, pega padrão comum. Alucina referência e falha com contexto fora do prompt. Útil como lente extra, nunca como árbitro.

A regra da dupla camada

O time maduro roda Sonar no pipeline, usa IA como lente extra, e reserva a energia do revisor humano para o que as duas camadas não pegam. Se o humano está repetindo comentário que Sonar ou IA pegariam, o time está desperdiçando o recurso mais caro no trabalho mais barato.

Checagem 1 de 2, rubrica de revisão

inteli.

Em um MR, o revisor comenta que prefere `const` em vez de `let` em uma variável que nunca é reatribuída. O Prettier e o ESLint do repo não exigem isso. Qual rótulo cabe melhor?

- Blocker, porque `const` expressa intenção e evita bug futuro
- Nit, o autor pode aceitar ou ignorar sem precisar justificar
- Question, por ser uma escolha que precisa de discussão na MR
- Não comente, isso é trabalho do lint

Checagem 2 de 2, IA na revisão

inteli.

A IA comenta em um MR: "A função `calc` deveria chamar `computeEligibilityScore`, que já existe em `src/shared/eligibility.js`, linha 42." Você abre o arquivo e ele não existe. Qual é a interpretação correta?

- A IA está certa, alguém apagou o arquivo em outro MR, reclamar no time
- Ignorar, IA nunca está certa de qualquer forma
- Alucinação, a IA inventou o arquivo, comentário descartado, padrão a observar no restante da revisão
- Abrir issue pedindo para o autor criar o arquivo que a IA citou

Auto-estudo para a Aula 05

inteli.

A Aula 05 abre o tema de **arquitetura e gabaritos de projeto**. Três provocações antes dela.

Leitura, C4 model

Simon Brown, primer do C4 em `c4model.com`. Foque nos quatro níveis de visão e por que a pirâmide funciona.

Vídeo, boundaries

Palestra curta sobre como times mal-colocados geram software mal-dividido. A lei de Conway de roupa nova.

Exercício, olhe seu repo

Desenhe em uma folha o diagrama de contexto do projeto do módulo: quem chama, quem é chamado, qual dado atravessa. 15 min, caneta de verdade.

Entrega desta aula

MR do exercício A com descrição, commits e testes. Tabela do exercício C com triagem dos alertas do Sonar. Tabela do exercício D com comparação humano vs IA. Os três pacotes vão para o AdaLove como diagnóstico da sprint, não como avaliação.

inteli.

Até a próxima

Aula 05, Arquitetura e gabaritos de projeto, 12/05